microgpt

The complete algorithm of a GPT in 200 lines of Python

Slides built on Andrej Karpathy's *microgpt* post, 12 Feb 2026

October 11, 2026

Outline

Orientation

Dataset

Tokenizer

Autograd

Parameters

Architecture

Training loop

Inference

Run it and build it

From microgpt to production

FAQ and summary

Orientation

What microgpt is

- One file, **200 lines of pure Python**, imports only `os`, `math`, `random`.
- Downloads 32,033 names, trains a tiny GPT for 1,000 steps, prints 20 invented names.
- Contains every *algorithmic* ingredient: dataset, tokenizer, autograd, GPT-2-style Transformer, Adam, training loop, inference loop.
- Culmination of micrograd $\rightarrow$ makemore $\rightarrow$ nanoGPT $\rightarrow$ llm.c (Karpathy 2020, 2022a,b, 2024b).

Karpathy's claim

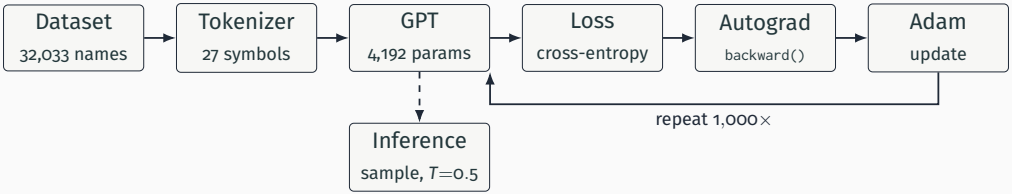
"Everything else is just efficiency. I cannot simplify this any further." (Karpathy 2026b)

Where to find it

- Blog post: `https://karpathy.github.io/2026/02/12/microgpt/`
- Gist `microgpt.py`: `https://gist.github.com/karpathy/8627fe009c40f57531cb18360106ce95`
- Step-by-step build `build_microgpt.py` (revisions `train0.py` … `train5.py`) (Karpathy 2026a)
- Colab notebook (Karpathy 2026c); web rendering at `https://karpathy.ai/microgpt.html`

`python microgpt.py` – nothing to install.

The pipeline



Notation: $V{=}27$ tokens, $d{=}16$ (`n_embd`), $H{=}4$ heads, $d_h{=}4$, $L{=}1$ layer, $T_{\max}{=}16$ (`block_size`).

Dataset

Dataset: a list of documents

```
if not os.path.exists('input.txt'):
    import urllib.request
    urllib.request.urlretrieve(names_url, 'input.txt')  # names.txt from the makemore repo
docs = [line.strip() for line in open('input.txt') if line.strip()]
random.shuffle(docs)
```

`emma, olivia, ava, isabella, sophia, charlotte, mia, amelia, harper, . . .`

32,033 names; longest is 15 letters.

- Each name is one *document*.
- Production: web pages, books, code, trillions of tokens.
- Nothing downstream depends on *what* the documents are.

A conversation is just a document

Key idea

From the model's point of view, a ChatGPT conversation is a single, oddly formatted document. The prompt is a prefix; the reply is a statistical *completion*, one token at a time.

- microgpt completing *k-a-r* $\rightarrow$ *karai* and ChatGPT completing a prompt into a paragraph are the same operation.
- Lineage: Shannon's letter-prediction experiments (Shannon 1951), neural LMs (Bengio et al. 2003), character-level RNNs (Graves 2013; Karpathy 2015; Sutskever et al. 2011).

Tokenizer

Tokenizer: characters as discrete symbols

```
uchars = sorted(set(''.join(docs))) # unique characters -> ids 0..n-1
BOS = len(uchars)                   # special Beginning of Sequence token
vocab_size = len(uchars) + 1        # 26 letters + BOS = 27
```

- a$\mapsto$ 0, . . . , z$\mapsto$ 25, BOS$\mapsto$ 26. Encode: `uchars.index(ch)`; decode: `uchars[id]`.
- Ids have **no arithmetic meaning**: “they might as well be different emoji”.
- `BOS` wraps every document: `[BOS, e, m, m, a, BOS]` – it starts a name and ends one.

Production tokenizers: byte-pair encoding

- BPE: start from bytes, repeatedly merge the most frequent adjacent pair (Gage 1994; Sennrich et al. 2016).
- GPT-2: ∼50K tokens; GPT-4 / tiktoken: ∼100K (OpenAI 2022; Radford et al. 2019).
- Common words become one token; rare words split into pieces ⇒ more content per position.
- SentencePiece is the other common implementation (Kudo and Richardson 2018).
- Next step after this course: Karpathy's tokenizer lecture (Karpathy 2024a).

Autograd

Why gradients

- Training adjusts each parameter θ to lower the loss $\mathcal{L}$: we need $\partial\mathcal{L}/\partial\theta$.
- “If I nudge this number up a little, does the loss go up or down, and by how much?”
- Thousands of inputs, **one** scalar output $\Rightarrow$ reverse-mode autodiff (backpropagation): one backward sweep gives all gradients.
- History: Linnainmaa (1976), Rumelhart et al. (1986); survey: Baydin et al. (2018).

The `Value` class: forward

```
class Value:
    __slots__ = ('data', 'grad', '_children', '_local_grads')
    def __init__(self, data, children=(), local_grads=()):
        self.data = data; self.grad = 0
        self._children = children; self._local_grads = local_grads
    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data + other.data, (self, other), (1, 1))
    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data * other.data, (self, other), (other.data, self.data))
    def __pow__(self, other): return Value(self.data**other, (self,), (other * self.data**(other-1),))
    def log(self):  return Value(math.log(self.data), (self,), (1/self.data,))
    def exp(self):  return Value(math.exp(self.data), (self,), (math.exp(self.data),))
    def relu(self): return Value(max(0, self.data), (self,), (float(self.data > 0),))
```

Each operation returns a new node remembering its inputs and its *local* derivative.

The six lego blocks

Operation	Forward	Local gradients
`a + b`	$a + b$	$1,\ 1$
`a * b`	ab	$b,\ a$
`a ** n`	a^n	na^{n-1}
`a.log()`	$\ln a$	$1/a$
`a.exp()`	e^a	e^a
`a.relu()`	$\max(0, a)$	$\mathbf{1}[a > 0]$

Subtraction, division and negation are derived: `a - b = a + (-1)*b`, `a / b = a * b**-1`.

The `Value` class: backward

```
def backward(self):
    topo, visited = [], set()
    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._children:
                build_topo(child)
            topo.append(v)
    build_topo(self)
    self.grad = 1
    for v in reversed(topo):
        for child, lg in zip(v._children, v.
     _local_grads):
            child.grad += lg * v.grad
```

1. **Topological sort** by DFS: children before parents.
2. **Seed** $\partial \mathcal{L}/\partial \mathcal{L} = 1$.
3. **Chain rule**, root to leaves:

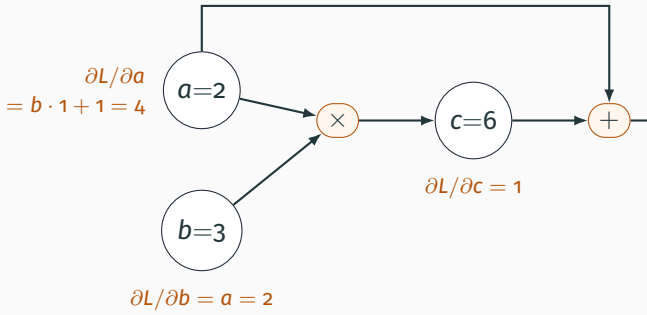
$$\frac{\partial \mathcal{L}}{\partial c} \mathrel{+}= \frac{\partial v}{\partial c} \cdot \frac{\partial \mathcal{L}}{\partial v}$$

The += sums contributions when a node feeds several parents (multivariable chain rule).

Worked example: $L = ab + a$

```
a = Value(2.0); b = Value(3.0)
c = a * b       # 6.0
L = c + a       # 8.0
L.backward()
print(a.grad)   # 4.0
print(b.grad)   # 2.0
```

PyTorch gives `tensor(4.)`, `tensor(2.)`: same algorithm, on tensors.



Meaning: raising a by 0.001 raises L by about 0.004. Sign = direction, magnitude = steepness.

Scalars vs. tensors

- microgpt: one graph node per scalar operation. One training step on "christopher" builds $\approx 10^5$ nodes.
- PyTorch (Paszke et al. 2017, 2019): one node per *tensor* operation; a backward node can be a fused GPU kernel.
- "Algorithmically identical, significantly smaller and simpler, but of course a lot less efficient."

Document	n	`Value` nodes	forward (s)	backward (s)
emma	5	39,703	0.055	0.038
isabella	9	73,263	0.161	0.069
christopher	12	99,483	0.251	0.099

Parameters

Parameters: the knowledge of the model

```
n_layer, n_embd, block_size, n_head = 1, 16, 16, 4
head_dim = n_embd // n_head
matrix = lambda nout, nin, std=0.08: [[Value(random.gauss(0, std)) for _ in range(nin)] for _ in range(nout)
    ]
state_dict = {'wte': matrix(vocab_size, n_embd), 'wpe': matrix(block_size, n_embd), 'lm_head': matrix(
    vocab_size, n_embd)}
for i in range(n_layer):
    state_dict[f'layer{i}.attn_wq'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wk'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wv'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wo'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc1'] = matrix(4 * n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc2'] = matrix(n_embd, 4 * n_embd)
params = [p for mat in state_dict.values() for row in mat for p in row]
```

Small Gaussian init ($\sigma = 0.08$); named matrices in a `state_dict`; flat list `params` for the optimizer.

Counting to 4,192

Name	Shape		Count
`wte`	$V \times d$	27×16	432
`wpe`	$T_{\max} \times d$	16×16	256
`lm_head`	$V \times d$	27×16	432
`attn_wq/wk/wv/wo`	$4\,(d \times d)$	4×256	1,024
`mlp_fc1`	$4d \times d$	64×16	1,024
`mlp_fc2`	$d \times 4d$	16×64	1,024
Total			**4,192**

Per layer: $12d^2$ (attention $4d^2$ + MLP $8d^2$). GPT-2: 1.5B; GPT-3: 175B (Brown et al. 2020; Radford et al. 2019). No biases, no weight tying (Press and Wolf 2017), init scale cf. Glorot and Bengio (2010).

Architecture

Three helpers

```python
def linear(x, w):            # matrix-vector product: one dot product per row of w
    return [sum(wi * xi for wi, xi in zip(wo, x)) for wo in w]

def softmax(logits):         # scores -> probabilities; subtract max for stability
    max_val = max(val.data for val in logits)
    exps = [(val - max_val).exp() for val in logits]
    total = sum(exps)
    return [e / total for e in exps]

def rmsnorm(x):              # rescale to unit root-mean-square (no learned gain)
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]
```

$$\text{softmax}(z)_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}, \qquad \text{rmsnorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \varepsilon}}$$ (Zhang and Sennrich 2019)

The model function (1/3): embeddings and attention

```
def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict['wte'][token_id]; pos_emb = state_dict['wpe'][pos_id]
    x = [t + p for t, p in zip(tok_emb, pos_emb)]
    x = rmsnorm(x) # note: not redundant due to backward pass via the residual connection
    for li in range(n_layer):
        # 1) Multi-head Attention block
        x_residual = x
        x = rmsnorm(x)
        q = linear(x, state_dict[f'layer{li}.attn_wq'])
        k = linear(x, state_dict[f'layer{li}.attn_wk'])
        v = linear(x, state_dict[f'layer{li}.attn_wv'])
        keys[li].append(k); values[li].append(v)
```

Embeddings, the first `rmsnorm`, the pre-norm, the q, k, v projections, and the cache append.

The model function (2/3): the head loop

```
        x_attn = []
        for h in range(n_head):
            hs = h * head_dim
            q_h = q[hs:hs+head_dim]
            k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
            v_h = [vi[hs:hs+head_dim] for vi in values[li]]
            attn_logits = [sum(q_h[j] * k_h[t][j] for j in range(head_dim)) / head_dim**0.5 for t in range(len(k_h))]
            attn_weights = softmax(attn_logits)
            head_out = [sum(attn_weights[t] * v_h[t][j] for t in range(len(v_h))) for j in range(head_dim)]
            x_attn.extend(head_out)
        x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
        x = [a + b for a, b in zip(x, x_residual)]
```

Per head: slice q, k, v; scaled dot products with every cached key; softmax; weighted sum of cached values. Concatenate heads, project with W_o, add the residual. GPT-2 (Radford et al. 2019) with: LayerNorm $\rightarrow$ RMSNorm, no biases, GeLU $\rightarrow$ ReLU.

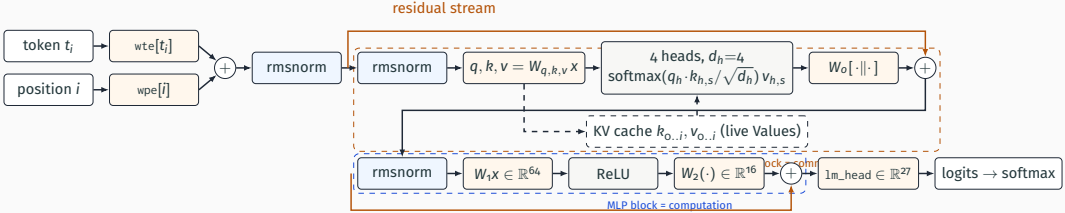
The model function (3/3): MLP and output

```
        # 2) MLP block
        x_residual = x
        x = rmsnorm(x)
        x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
        x = [xi.relu() for xi in x]
        x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
        x = [a + b for a, b in zip(x, x_residual)]

    logits = linear(x, state_dict['lm_head'])
    return logits
```

- Each block: $x \leftarrow x + f(\text{rmsnorm}(x))$, the *pre-norm* residual pattern of GPT-2.
- MLP: $W_2\ \text{relu}(W_1 x)$ with $W_1 \in \mathbb{R}^{64\times 16}$, $W_2 \in \mathbb{R}^{16\times 64}$; fully local to one position.
- `lm_head` maps the final $x \in \mathbb{R}^{16}$ to 27 logits. Softmax is applied by the caller.

The architecture, one token at a time



Orange = parameters, blue = parameter-free normalisation, orange arrows = residual connections. For $L > 1$ the two blocks repeat.

Embeddings

$$x_i^{(0)} = \text{rmsnorm}\left(\text{wte}[t_i] + \text{wpe}[i]\right) \in \mathbb{R}^{16}$$

- A network cannot use the symbol “5”; it looks up a learned vector per token and per position and adds them: *what* + *where*.
- Attention is a weighted sum over cached positions, so `wpe` is the *only* source of order information.
- Original Transformer: fixed sinusoids (Vaswani et al. 2017); GPT-1/2: learned table; modern: RoPE, relative rotations of q, k (Su et al. 2021).
- The first `rmsnorm` fixes the scale at which the residual stream starts; pre-norm inside blocks only touches the branch input.

Attention

$$q = W_q x, \quad k = W_k x, \quad v = W_v x$$

$$a_{h,s} = \frac{q[S_h] \cdot k_s[S_h]}{\sqrt{d_h}}, \qquad s \le i$$

$$\alpha_h = \mathrm{softmax}(a_{h,0}, \ldots, a_{h,i})$$

$$\mathrm{head}_h = \textstyle\sum_{s \le i} \alpha_{h,s}\, v_s[S_h]$$

$$\mathrm{out} = W_o\, [\mathrm{head}_1 \| \cdots \| \mathrm{head}_4]$$

- Query “what am I looking for?”, key “what do I contain?”, value “what do I offer?”
- k, v appended to the cache each step.
- $1/\sqrt{d_h}$ keeps logits at unit variance (Vaswani et al. 2017).
- 4 heads $\times$ width 4 cost the same as 1 head $\times$ 16; they buy *diversity*.
- **No causal mask needed**: the cache holds only $s \le i$.

Attention is communication; MLP is computation

- The attention block is the **only** place where position i reads from positions $0..i-1$.
- MLP: $W_2\ \mathrm{relu}(W_1 x)$, project up $4\times$, nonlinearity, project down. Fully local to one position: "where the model does most of its thinking".
- GPT-2 used GELU (Hendrycks and Gimpel 2016); modern models use gated SwiGLU (Shazeer 2020; Touvron et al. 2023) or squared ReLU (So et al. 2021).
- Residuals $x \leftarrow x + f(\mathrm{rmsnorm}(x))$ let gradients flow straight through (He et al. 2016); pre-norm stabilises training (Xiong et al. 2020).
- The "residual stream" view: blocks read from and write to a shared vector (Elhage et al. 2021).

The KV cache, even during training

- Usually associated with inference; Karpathy: it is “conceptually always there”.
- Vectorised training hides it inside one big attention matmul. microgpt processes one token at a time, so the cache is explicit.
- Here cached k, v are **live** `Value` nodes: loss at position 7 backpropagates into the parameters that produced k_2, v_2.
- Cache size $2\,L\,T\,d$ per sequence motivates MQA/GQA (Ainslie et al. 2023; Shazeer 2019) and paged caches in vLLM (Kwon et al. 2023); accounting in Pope et al. (2022).

Cost per token $\approx 12d^2 + Vd + O(i\,d)$ multiply–adds $\approx 2\times$(non-embedding params): the $2N$ / $6N$ rule of Kaplan et al. (2020) already holds.

Training loop

Training loop: pick a document, forward, loss, backward, update

```
learning_rate, beta1, beta2, eps_adam = 0.01, 0.85, 0.99, 1e-8
m = [0.0] * len(params); v = [0.0] * len(params); num_steps = 1000
for step in range(num_steps):
    doc = docs[step % len(docs)]
    tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
    n = min(block_size, len(tokens) - 1)
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses)   # May yours be low.
    loss.backward()
    lr_t = learning_rate * (1 - step / num_steps)   # linear decay
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1)); v_hat = v[i] / (1 - beta2 ** (step + 1))
        p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
        p.grad = 0
```

Next-token prediction and cross-entropy

"emma" $\to$ `[BOS, e, m, m, a, BOS]`; position i sees $t_{\le i}$, predicts t_{i+1}; $n = 5$ predictions.

$$\ell_i = -\log p(t_{i+1} \mid t_{\le i}), \qquad \mathcal{L} = \frac{1}{n}\sum_{i=0}^{n-1} \ell_i$$

Three views of $-\log p$

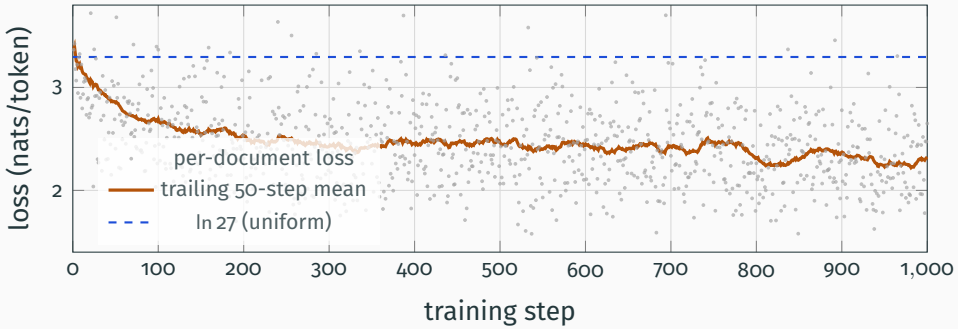
- Surprise / surprisal (Shannon 1948)
- Maximum likelihood of the corpus
- Cross-entropy = entropy + KL (Goodfellow et al. 2016)

Reference values

- Uniform guess: $\ln 27 \approx 3.30$
- First step: 3.37; after 1,000 steps: ≈ 2.3
- Perplexity $e^{\mathcal{L}}$: 27 $\to \approx$ 10
- Bits/char $\mathcal{L}/\ln 2$: 4.75 $\to \approx$ 3.4

Bonus identity (derived automatically by autograd): $\partial \ell / \partial z_k = p_k - y_k$.

The loss curve (our run, seed 42)



Batch size one ⇒ noisy dots. Only 1,000 of 32,033 names seen: every step is on fresh data, so this is effectively a held-out loss.

Adam

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad \hat{m}_t = m_t / (1-\beta_1^t)$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \qquad \hat{v}_t = v_t / (1-\beta_2^t)$$
$$\theta_t = \theta_{t-1} - \eta_t \, \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

- m: momentum (Polyak 1964); v: per-parameter step size (RMSProp, Tieleman and Hinton 2012); $\hat{m}, \hat{v}$: bias corrections (Kingma and Ba 2015).
- microgpt: β_1=0.85, β_2=0.99 (defaults 0.9, 0.999); $\eta_t = 0.01\,(1 - t/1000)$.
- At scale: AdamW + warmup + cosine (Loshchilov and Hutter 2017, 2019). Reset `.grad = 0` after each step.

Inference

Inference: sampling

```
temperature = 0.5
for sample_idx in range(20):
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    token_id = BOS
    sample = []
    for pos_id in range(block_size):
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax([l / temperature for l in logits])
        token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
        if token_id == BOS:
            break
        sample.append(uchars[token_id])
    print(f"sample {sample_idx+1:2d}: {''.join(sample)}")
```

Start from `BOS`, sample a token, feed it back, stop at `BOS` or $T_{\max}$: autoregressive (ancestral) sampling.

Temperature

$$p_k^{(T)} = \frac{e^{z_k/T}}{\sum_j e^{z_j/T}}$$

- $T = 1$: the learned distribution.
- $T < 1$: sharper, conservative (0.5 here).
- $T \to 0$: greedy decoding (arg-max).
- $T > 1$: flatter, more diverse, less coherent.

- Origin: Boltzmann distribution (Ackley et al. 1985); see also Hinton et al. (2015).
- Companions: top-k (Fan et al. 2018), nucleus sampling (Holtzman et al. 2020).

Samples (seed 42): kamon, ann, karai, jaire, vialan, karia, yeran, anna, areli, kaina, konna, keylen, liole, alerin, earan, lenne, kana, lara, alela, anton. Plausible, mostly *not* in the training set: “hallucination”.

Run it and build it

Run it

```
python microgpt.py
```

```
num docs: 32033
vocab size: 27
num params: 4192
step    1 / 1000 | loss 3.3660
...
step 1000 / 1000 | loss 2.6497
--- inference (new, hallucinated names) ---
sample  1: kamon
```

- Karpathy: about 1 minute on his MacBook. Our laptop: 3 min 24 s in CPython.
- Knobs: `num_steps`, `n_embd`, `n_layer`, `n_head`, `temperature`, the dataset file.

Progression: layers of the onion

File	What it adds
`train0.py`	Bigram count table: no neural net, no gradients
`train1.py`	MLP + manual gradients (numerical & analytic) + SGD
`train2.py`	Autograd (`Value` class) replaces manual gradients
`train3.py`	Position embeddings + single-head attention + rmsnorm + residuals
`train4.py`	Multi-head attention + layer loop: full GPT
`train5.py`	Adam optimizer: this is `train.py`

Revisions of `build_microgpt.py` (Karpathy 2026a). Mirrors history: n-grams $\rightarrow$ neural LMs $\rightarrow$ backprop $\rightarrow$ attention $\rightarrow$ adaptive optimizers.

From microgpt to production

What changes at scale (nothing in the algorithm)

	microgpt	Production
Data	32K names	Trillions of tokens; dedup, filtering, mixing
Tokenizer	27 chars	Byte-level BPE, $\sim$100K tokens
Autograd	scalar `Value`	Tensors on GPUs; fused kernels (FlashAttention)
Architecture	4,192 params, $L{=}1$	10^{11+} params, $L{>}100$; RoPE, GQA, gated MLP, MoE
Training	1 doc/step	Millions of tokens/step, mixed precision, months on thousands of GPUs
Optimisation	Adam, linear decay	Tuned AdamW, warmup, scaling laws
Post-training	none	SFT then RL
Inference	1 sample at a time	Batching, paged KV, speculative decoding, quantisation

Data, tokenizer, hardware

- **Data**: web, books, code; deduplication measurably helps (Lee et al. 2022); curated corpora (Gao et al. 2020; Penedo et al. 2024; Raffel et al. 2020).
- **Tokenizer**: BPE gives $\sim$4 characters per token in English, multiplying effective context.
- **Autograd**: same maths over tensors (Paszke et al. 2019); FlashAttention tiles attention through on-chip memory without materialising the $T \times T$ matrix (Dao 2024; Dao et al. 2022).

Architecture and training at scale

- Residual stream of attention + MLP blocks is preserved.
- Substitutions: RoPE (Su et al. 2021), grouped-query attention (Ainslie et al. 2023), SwiGLU (Shazeer 2020), Mixture of Experts (DeepSeek-AI 2024; Fedus et al. 2022; Shazeer et al. 2017). Pseudocode for all variants: Phuong and Hutter (2022).
- Batches of millions of tokens, gradient accumulation, bf16/fp8 (Micikevicius et al. 2018).
- Optimisation is its own discipline: LR, warmup, decay, weight decay, betas all interact with size.
- Scaling laws (Kaplan et al. 2020); Chinchilla: $\approx$ 20 tokens per parameter (Hoffmann et al. 2022).

Post-training and serving

- Pretrained model = document completer, not a chatbot.
- **SFT**: swap documents for conversations, keep training. Algorithmically nothing changes.
- **RL**: model generates, outputs are scored, model updates toward high scores (Christiano et al. 2017; Ouyang et al. 2022; Rafailov et al. 2023; Schulman et al. 2017; Shao et al. 2024). “Still training on documents, but written by the model itself.”
- **Serving**: continuous batching (Yu et al. 2022), PagedAttention (Kwon et al. 2023), speculative decoding (Chen et al. 2023; Leviathan et al. 2023), int8/int4 quantisation (Dettmers et al. 2022; Frantar et al. 2023).
- End-to-end at small scale: nanochat (Karpathy 2025).

FAQ and summary

FAQ (from the post)

Understands? Mechanically, no magic: a function from tokens to a next-token distribution; the mechanism is all in the 200 lines.

Why does it work? Small nudges, many steps; parameters settle into the statistics of the data (consonant starts, “qu”, no triple consonants).

Relation to ChatGPT? Same loop, scaled up, plus post-training. Prompt and reply are tokens in one sequence.

Hallucinations? Samples from a distribution over plausible sequences; no concept of truth. *karia* $\equiv$ a confident false fact.

Why slow? One scalar at a time in Python. GPUs: millions in parallel.

Better names? Train longer, bigger model, more data: the same three knobs as at scale.

Other data? Learns whatever is in the file; nothing else changes.

Summary

1. **Dataset**: text as documents; a chat is a document.
2. **Tokenizer**: 27 symbols; BOS delimits documents.
3. **Autograd**: six local derivatives + topological sort + chain rule with +=.
4. **Parameters**: 4,192 Gaussian numbers in named matrices.
5. **Architecture**: embeddings $\rightarrow$ [rmsnorm, attention, residual] $\rightarrow$ [rmsnorm, MLP, residual] $\rightarrow$ logits. Attention communicates; MLP computes.
6. **Training**: cross-entropy on the next token, backward, Adam with linear decay. 3.3 $\rightarrow$ 2.3 nats.
7. **Inference**: sample with temperature, feed back, stop at BOS.

"If you understand microgpt, you understand the algorithmic essence."

Exercises to take home

- Derive 4,192 by hand; generalise to GPT-2 small.
- Gradient-check one parameter with finite differences.
- Build `train0.py` (bigram counts) and compare its loss with ≈ 2.3.
- Replace Adam with SGD; try cosine decay; try minibatches of 8 names.
- Hold out 10% of names and watch for a train/validation gap at 50,000 steps.
- Implement greedy, top-k and nucleus decoding.
- Swap ReLU for squared ReLU; tie `lm_head` to `wte`; add RoPE.
- Train on a different list of short strings.

Full exercise list and solutions sketches: the lecture notes and the study guide.

References i

Ackley, D. H., G. E. Hinton, and T. J. Sejnowski (1985). **"A Learning Algorithm for Boltzmann Machines".** In: *Cognitive Science* 9.1. Origin of the "temperature" softmax in neural networks, pp. 147–169.

Ainslie, J., J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai (2023). **"GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints".** In: *Proceedings of EMNLP*. arXiv:2305.13245.

Baydin, A. G., B. A. Pearlmutter, A. A. Radul, and J. M. Siskind (2018). **"Automatic Differentiation in Machine Learning: a Survey".** In: *Journal of Machine Learning Research* 18.153, pp. 1–43.

Bengio, Y., R. Ducharme, P. Vincent, and C. Jauvin (2003). **"A Neural Probabilistic Language Model".** In: *Journal of Machine Learning Research* 3, pp. 1137–1155.

Brown, T. B. et al. (2020). **"Language Models are Few-Shot Learners".** In: *Advances in Neural Information Processing Systems 33 (NeurIPS)*. arXiv:2005.14165.

References ii

Chen, C., S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper (2023). **"Accelerating Large Language Model Decoding with Speculative Sampling".** In: *arXiv preprint arXiv:2302.01318*.

Christiano, P. F., J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei (2017). **"Deep Reinforcement Learning from Human Preferences".** In: *Advances in Neural Information Processing Systems 30 (NIPS)*. arXiv:1706.03741.

Dao, T. (2024). **"FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning".** In: *International Conference on Learning Representations (ICLR)*. arXiv:2307.08691.

Dao, T., D. Y. Fu, S. Ermon, A. Rudra, and C. Ré (2022). **"FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness".** In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2205.14135.

DeepSeek-AI (2024). **"DeepSeek-V3 Technical Report".** In: *arXiv preprint arXiv:2412.19437*. Large-scale MoE, multi-head latent attention, FP8 training.

References iii

Dettmers, T., M. Lewis, Y. Belkada, and L. Zettlemoyer (2022). **“LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale”.** In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2208.07339.

Elhage, N. et al. (2021). ***A Mathematical Framework for Transformer Circuits.*** The “residual stream” view of the Transformer. URL: https://transformer-circuits.pub/2021/framework/index.html (visited on 10/11/2026).

Fan, A., M. Lewis, and Y. Dauphin (2018). **“Hierarchical Neural Story Generation”.** In: *Proceedings of ACL*. arXiv:1805.04833; top-k sampling.

Fedus, W., B. Zoph, and N. Shazeer (2022). **“Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity”.** In: *Journal of Machine Learning Research* 23.120, pp. 1–39.

Frantar, E., S. Ashkboos, T. Hoefler, and D. Alistarh (2023). **“GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers”.** In: *International Conference on Learning Representations (ICLR)*. arXiv:2210.17323.

References iv

Gage, P. (1994). **“A New Algorithm for Data Compression”.** In: *C Users Journal* 12.2, pp. 23–38.

Gao, L. et al. (2020). **“The Pile: An 800GB Dataset of Diverse Text for Language Modeling”.** In: *arXiv preprint arXiv:2101.00027*.

Glorot, X. and Y. Bengio (2010). **“Understanding the Difficulty of Training Deep Feedforward Neural Networks”.** In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*.

Goodfellow, I., Y. Bengio, and A. Courville (2016). ***Deep Learning*.** Free online at `https://www.deeplearningbook.org`. MIT Press.

Graves, A. (2013). **“Generating Sequences With Recurrent Neural Networks”.** In: *arXiv preprint arXiv:1308.0850*.

He, K., X. Zhang, S. Ren, and J. Sun (2016). **“Deep Residual Learning for Image Recognition”.** In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:1512.03385, pp. 770–778.

Hendrycks, D. and K. Gimpel (2016). **“Gaussian Error Linear Units (GELUs)”.** In: *arXiv preprint arXiv:1606.08415*.

Hinton, G., O. Vinyals, and J. Dean (2015). **“Distilling the Knowledge in a Neural Network”.** In: *arXiv preprint arXiv:1503.02531*. Clear discussion of softmax temperature.

Hoffmann, J. et al. (2022). **“Training Compute-Optimal Large Language Models”.** In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2203.15556; the “Chinchilla” scaling laws.

Holtzman, A., J. Buys, L. Du, M. Forbes, and Y. Choi (2020). **“The Curious Case of Neural Text Degeneration”.** In: *International Conference on Learning Representations (ICLR)*. arXiv:1904.09751; nucleus (top-p) sampling.

Kaplan, J. et al. (2020). **“Scaling Laws for Neural Language Models”.** In: *arXiv preprint arXiv:2001.08361*.

Karpathy, A. (2015). ***The Unreasonable Effectiveness of Recurrent Neural Networks.*** URL: https://karpathy.github.io/2015/05/21/rnn-effectiveness/ (visited on 10/11/2026).

Karpathy, A. (2020). ***micrograd: a tiny scalar-valued autograd engine and a neural net library on top of it.*** URL: https://github.com/karpathy/micrograd (visited on 10/11/2026).

Karpathy, A. (2022a). ***makemore: an autoregressive character-level language model for making more things.*** Source of the names.txt dataset used by microgpt. URL: https://github.com/karpathy/makemore (visited on 10/11/2026).

Karpathy, A. (2022b). ***nanoGPT: the simplest, fastest repository for training/finetuning medium-sized GPTs.*** URL: https://github.com/karpathy/nanoGPT (visited on 10/11/2026).

Karpathy, A. (2024a). ***Let's build the GPT Tokenizer.*** URL: https://www.youtube.com/watch?v=zduSFxRajkE (visited on 10/11/2026).

Karpathy, A. (2024b). ***llm.c: LLM training in simple, raw C/CUDA.*** URL: https://github.com/karpathy/llm.c (visited on 10/11/2026).

References vii

Karpathy, A. (2025). ***nanochat: the best ChatGPT that $100 can buy.*** Full pretraining, SFT, RL and serving pipeline in one small repo. URL: https://github.com/karpathy/nanochat (visited on 10/11/2026).

Karpathy, A. (2026a). ***build_microgpt.py: builds up microgpt.py piece by piece.*** GitHub gist; the Revisions tab holds train0.py through train5.py. URL: https://gist.github.com/karpathy/561ac2de12a47cc06a23691e1be9543a (visited on 10/11/2026).

Karpathy, A. (Feb. 2026b). ***microgpt.*** Blog post; the primary source for this course. URL: https://karpathy.github.io/2026/02/12/microgpt/ (visited on 10/11/2026).

Karpathy, A. (2026c). ***microgpt Google Colab notebook.*** URL: https://colab.research.google.com/drive/1vyN5zo6rqUp_dYNbT4Yrco66zuWCZKoN (visited on 10/11/2026).

Kingma, D. P. and J. Ba (2015). **"Adam: A Method for Stochastic Optimization".** In: *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.

Kudo, T. and J. Richardson (2018). **“SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing”.** In: *Proceedings of EMNLP: System Demonstrations*. arXiv:1808.06226.

Kwon, W. et al. (2023). **“Efficient Memory Management for Large Language Model Serving with PagedAttention”.** In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. arXiv:2309.06180; vLLM.

Lee, K. et al. (2022). **“Deduplicating Training Data Makes Language Models Better”.** In: *Proceedings of ACL*. arXiv:2107.06499.

Leviathan, Y., M. Kalman, and Y. Matias (2023). **“Fast Inference from Transformers via Speculative Decoding”.** In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. arXiv:2211.17192.

Linnainmaa, S. (1976). **“Taylor Expansion of the Accumulated Rounding Error”.** In: *BIT Numerical Mathematics* 16.2. The earliest published form of reverse-mode automatic differentiation, pp. 146–160.

References ix

Loshchilov, I. and F. Hutter (2017). **“SGDR: Stochastic Gradient Descent with Warm Restarts”.** In: *International Conference on Learning Representations (ICLR)*. arXiv:1608.03983; cosine learning-rate schedules.

Loshchilov, I. and F. Hutter (2019). **“Decoupled Weight Decay Regularization”.** In: *International Conference on Learning Representations (ICLR)*. arXiv:1711.05101; AdamW.

Micikevicius, P. et al. (2018). **“Mixed Precision Training”.** In: *International Conference on Learning Representations (ICLR)*. arXiv:1710.03740.

OpenAI (2022). ***tiktoken: a fast BPE tokeniser for use with OpenAI’s models.*** URL: `https://github.com/openai/tiktoken` (visited on 10/11/2026).

Ouyang, L. et al. (2022). **“Training Language Models to Follow Instructions with Human Feedback”.** In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2203.02155; InstructGPT: SFT + RLHF.

Paszke, A. et al. (2017). **“Automatic Differentiation in PyTorch”.** In: *NIPS 2017 Autodiff Workshop*.

Paszke, A. et al. (2019). **“PyTorch: An Imperative Style, High-Performance Deep Learning Library”.** In: *Advances in Neural Information Processing Systems 32 (NeurIPS)*.

Penedo, G. et al. (2024). **“The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale”.** In: *arXiv preprint arXiv:2406.17557*.

Phuong, M. and M. Hutter (2022). **“Formal Algorithms for Transformers”.** In: *arXiv preprint arXiv:2207.09238*. Pseudocode for every Transformer variant; an excellent companion to the code.

Polyak, B. T. (1964). **“Some Methods of Speeding Up the Convergence of Iteration Methods”.** In: *USSR Computational Mathematics and Mathematical Physics* 4.5, pp. 1–17.

Pope, R. et al. (2022). **“Efficiently Scaling Transformer Inference”.** In: *arXiv preprint arXiv:2211.05102*. Careful accounting of KV-cache memory and bandwidth.

Press, O. and L. Wolf (2017). **“Using the Output Embedding to Improve Language Models”.** In: *Proceedings of EACL*. arXiv:1608.05859; microgpt deliberately does *not* tie wte and lm_head.

Radford, A., J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever (2019). ***Language Models are Unsupervised Multitask Learners.*** Tech. rep. The GPT-2 report; microgpt follows this architecture. OpenAI.

Rafailov, R., A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn (2023). **“Direct Preference Optimization: Your Language Model is Secretly a Reward Model”.** In: *Advances in Neural Information Processing Systems 36 (NeurIPS)*. arXiv:2305.18290.

Raffel, C. et al. (2020). **“Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”.** In: *Journal of Machine Learning Research* 21.140. Introduces the C4 web corpus, pp. 1–67.

Rumelhart, D. E., G. E. Hinton, and R. J. Williams (1986). **“Learning Representations by Back-Propagating Errors”.** In: *Nature* 323, pp. 533–536.

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov (2017). **"Proximal Policy Optimization Algorithms".** In: *arXiv preprint arXiv:1707.06347*.

Sennrich, R., B. Haddow, and A. Birch (2016). **"Neural Machine Translation of Rare Words with Subword Units".** In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:1508.07909, pp. 1715–1725.

Shannon, C. E. (1948). **"A Mathematical Theory of Communication".** In: *Bell System Technical Journal* 27.3, pp. 379–423.

Shannon, C. E. (1951). **"Prediction and Entropy of Printed English".** In: *Bell System Technical Journal* 30.1, pp. 50–64.

Shao, Z. et al. (2024). **"DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models".** In: *arXiv preprint arXiv:2402.03300*. Introduces GRPO.

Shazeer, N. (2019). **"Fast Transformer Decoding: One Write-Head is All You Need".** In: *arXiv preprint arXiv:1911.02150*. Multi-Query Attention.

Shazeer, N. (2020). **“GLU Variants Improve Transformer”.** In: *arXiv preprint arXiv:2002.05202*.

Shazeer, N. et al. (2017). **“Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”.** In: *International Conference on Learning Representations (ICLR)*. arXiv:1701.06538.

So, D. R., W. Mańke, H. Liu, Z. Dai, N. Shazeer, and Q. V. Le (2021). **“Primer: Searching for Efficient Transformers for Language Modeling”.** In: *arXiv preprint arXiv:2109.08668*. Introduces squared ReLU in the Transformer MLP.

Su, J., Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu (2021). **“RoFormer: Enhanced Transformer with Rotary Position Embedding”.** In: *arXiv preprint arXiv:2104.09864*.

Sutskever, I., J. Martens, and G. Hinton (2011). **“Generating Text with Recurrent Neural Networks”.** In: *Proceedings of the 28th International Conference on Machine Learning (ICML)*.

References xiv

Tieleman, T. and G. Hinton (2012). ***Lecture 6.5 – RMSProp: Divide the Gradient by a Running Average of its Recent Magnitude.*** COURSERA: Neural Networks for Machine Learning.

Touvron, H. et al. (2023). **"LLaMA: Open and Efficient Foundation Language Models".** In: *arXiv preprint arXiv:2302.13971*. Popularised the RMSNorm + SwiGLU + RoPE recipe.

Vaswani, A. et al. (2017). **"Attention Is All You Need".** In: *Advances in Neural Information Processing Systems 30 (NIPS)*. arXiv:1706.03762.

Xiong, R. et al. (2020). **"On Layer Normalization in the Transformer Architecture".** In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*. arXiv:2002.04745; the Pre-LN arrangement used by GPT-2 and microgpt.

Yu, G.-I., J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun (2022). **"Orca: A Distributed Serving System for Transformer-Based Generative Models".** In: *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Continuous (iteration-level) batching.

Zhang, B. and R. Sennrich (2019). **“Root Mean Square Layer Normalization”.** In: *Advances in Neural Information Processing Systems 32 (NeurIPS)*. arXiv:1910.07467.